



[Overview](#) [Programming Guides ▾](#) [API Docs ▾](#) [Deploying ▾](#) [More ▾](#)

Spark Programming Guide

- [Overview](#)
- [Linking with Spark](#)
- [Initializing Spark](#)
 - [Using the Shell](#)
- [Resilient Distributed Datasets \(RDDs\)](#)
 - [Parallelized Collections](#)
 - [External Datasets](#)
 - [RDD Operations](#)
 - [Basics](#)
 - [Passing Functions to Spark](#)
 - [Working with Key-Value Pairs](#)
 - [Transformations](#)
 - [Actions](#)
 - [RDD Persistence](#)
 - [Which Storage Level to Choose?](#)
 - [Removing Data](#)
- [Shared Variables](#)
 - [Broadcast Variables](#)
 - [Accumulators](#)
- [Deploying to a Cluster](#)
- [Unit Testing](#)
- [Migrating from pre-1.0 Versions of Spark](#)
- [Where to Go from Here](#)

Overview

At a high level, every Spark application consists of a *driver program* that runs the user's main function and executes various *parallel operations* on a cluster. The main abstraction Spark provides is a *resilient distributed dataset* (RDD), which is a collection of elements partitioned across the nodes of the cluster that can be operated on in parallel. RDDs are created by starting with a file in the Hadoop file system (or any other Hadoop-supported file system), or an existing Scala collection in the driver program, and transforming it. Users may also ask Spark to *persist* an RDD in memory, allowing it to be reused efficiently across parallel operations. Finally, RDDs automatically recover from node failures.

A second abstraction in Spark is *shared variables* that can be used in parallel operations. By default, when Spark runs a function in parallel as a set of tasks on different nodes, it ships a copy of each variable used in the function to each task. Sometimes, a variable needs to be shared across tasks, or between tasks and the driver program. Spark supports two types of shared variables: *broadcast variables*, which can be used to cache a value in memory on all nodes, and *accumulators*, which are

variables that are only “added” to, such as counters and sums.

This guide shows each of these features in each of Spark’s supported languages. It is easiest to follow along with if you launch Spark’s interactive shell – either `bin/spark-shell` for the Scala shell or `bin/pyspark` for the Python one.

Linking with Spark

Scala

Java

Python

Spark 1.2.0 uses Scala 2.10. To write applications in Scala, you will need to use a compatible Scala version (e.g. 2.10.X).

To write a Spark application, you need to add a Maven dependency on Spark. Spark is available through Maven Central at:

```
groupId = org.apache.spark
artifactId = spark-core_2.10
version = 1.2.0
```

In addition, if you wish to access an HDFS cluster, you need to add a dependency on `hadoop-client` for your version of HDFS. Some common HDFS version tags are listed on the [third party distributions](#) page.

```
groupId = org.apache.hadoop
artifactId = hadoop-client
version = <your-hdfs-version>
```

Finally, you need to import some Spark classes and implicit conversions into your program. Add the following lines:

```
import org.apache.spark.SparkContext
import org.apache.spark.SparkContext._
import org.apache.spark.SparkConf
```

Initializing Spark

Scala

Java

Python

The first thing a Spark program must do is to create a [SparkContext](#) object, which tells Spark how to access a cluster. To create a `sparkContext` you first need to build a [SparkConf](#) object that contains information about your application.

Only one `SparkContext` may be active per JVM. You must `stop()` the active `SparkContext` before creating a new one.

```
val conf = new SparkConf().setAppName(appName).setMaster(master)
new SparkContext(conf)
```

The `appName` parameter is a name for your application to show on the cluster UI. `master` is a [Spark, Mesos or YARN cluster URL](#), or a special “local” string to run in local mode. In practice, when running on a cluster, you will not want to hardcode `master` in the program, but rather [launch the application with `spark-submit`](#) and receive it there. However, for local testing and unit tests, you can pass “local” to run Spark in-process.

Using the Shell

[Scala](#)[Python](#)

In the Spark shell, a special interpreter-aware `SparkContext` is already created for you, in the variable called `sc`. Making your own `SparkContext` will not work. You can set which master the context connects to using the `--master` argument, and you can add JARs to the classpath by passing a comma-separated list to the `--jars` argument. For example, to run `bin/spark-shell` on exactly four cores, use:

```
$ ./bin/spark-shell --master local[4]
```

Or, to also add `code.jar` to its classpath, use:

```
$ ./bin/spark-shell --master local[4] --jars code.jar
```

For a complete list of options, run `spark-shell --help`. Behind the scenes, `spark-shell` invokes the more general [spark-submit script](#).

Resilient Distributed Datasets (RDDs)

Spark revolves around the concept of a *resilient distributed dataset* (RDD), which is a fault-tolerant collection of elements that can be operated on in parallel. There are two ways to create RDDs: *parallelizing* an existing collection in your driver program, or referencing a dataset in an external storage system, such as a shared filesystem, HDFS, HBase, or any data source offering a Hadoop `InputFormat`.

Parallelized Collections

[Scala](#)[Java](#)[Python](#)

Parallelized collections are created by calling `sparkContext`’s `parallelize` method on an existing collection in your driver program (a Scala `Seq`). The elements of the collection are copied to form a distributed dataset that can be operated on in parallel. For example, here is how to create a

parallelized collection holding the numbers 1 to 5:

```
val data = Array(1, 2, 3, 4, 5)
val distData = sc.parallelize(data)
```

Once created, the distributed dataset (`distData`) can be operated on in parallel. For example, we might call `distData.reduce((a, b) => a + b)` to add up the elements of the array. We describe operations on distributed datasets later on.

One important parameter for parallel collections is the number of *partitions* to cut the dataset into. Spark will run one task for each partition of the cluster. Typically you want 2-4 partitions for each CPU in your cluster. Normally, Spark tries to set the number of partitions automatically based on your cluster. However, you can also set it manually by passing it as a second parameter to `parallelize` (e.g. `sc.parallelize(data, 10)`). Note: some places in the code use the term *slices* (a synonym for partitions) to maintain backward compatibility.

External Datasets

[Scala](#)[Java](#)[Python](#)

Spark can create distributed datasets from any storage source supported by Hadoop, including your local file system, HDFS, Cassandra, HBase, [Amazon S3](#), etc. Spark supports text files, [SequenceFiles](#), and any other Hadoop [InputFormat](#).

Text file RDDs can be created using `sparkContext`'s `textFile` method. This method takes an URI for the file (either a local path on the machine, or a `hdfs://`, `s3n://`, etc URI) and reads it as a collection of lines. Here is an example invocation:

```
scala> val distFile = sc.textFile("data.txt")
distFile: RDD[String] = MappedRDD@1d4cee08
```

Once created, `distFile` can be acted on by dataset operations. For example, we can add up the sizes of all the lines using the `map` and `reduce` operations as follows: `distFile.map(s => s.length).reduce((a, b) => a + b)`.

Some notes on reading files with Spark:

- If using a path on the local filesystem, the file must also be accessible at the same path on worker nodes. Either copy the file to all workers or use a network-mounted shared file system.
- All of Spark's file-based input methods, including `textFile`, support running on directories, compressed files, and wildcards as well. For example, you can use `textFile("/my/directory")`, `textFile("/my/directory/*.txt")`, and `textFile("/my/directory/*.gz")`.
- The `textFile` method also takes an optional second argument for controlling the number of partitions of the file. By default, Spark creates one partition for each block of the file (blocks being 64MB by default in HDFS), but you can also ask for a higher number of partitions by passing a larger value. Note that you cannot have fewer partitions than blocks.

Apart from text files, Spark's Scala API also supports several other data formats:

- `sparkContext.wholeTextFiles` lets you read a directory containing multiple small text files, and returns each of them as (filename, content) pairs. This is in contrast with `textFile`, which would return one record per line in each file.
- For [SequenceFiles](#), use SparkContext's `sequenceFile[K, V]` method where `K` and `V` are the types of key and values in the file. These should be subclasses of Hadoop's [Writable](#) interface, like [IntWritable](#) and [Text](#). In addition, Spark allows you to specify native types for a few common Writables; for example, `sequenceFile[Int, String]` will automatically read `IntWritable`s and `Text`s.
- For other Hadoop InputFormats, you can use the `sparkContext.hadoopRDD` method, which takes an arbitrary `JobConf` and input format class, key class and value class. Set these the same way you would for a Hadoop job with your input source. You can also use `sparkContext.newHadoopRDD` for InputFormats based on the “new” MapReduce API (`org.apache.hadoop.mapreduce`).
- `RDD.saveAsObjectFile` and `sparkContext.objectFile` support saving an RDD in a simple format consisting of serialized Java objects. While this is not as efficient as specialized formats like Avro, it offers an easy way to save any RDD.

RDD Operations

RDDs support two types of operations: *transformations*, which create a new dataset from an existing one, and *actions*, which return a value to the driver program after running a computation on the dataset. For example, `map` is a transformation that passes each dataset element through a function and returns a new RDD representing the results. On the other hand, `reduce` is an action that aggregates all the elements of the RDD using some function and returns the final result to the driver program (although there is also a parallel `reduceByKey` that returns a distributed dataset).

All transformations in Spark are *lazy*, in that they do not compute their results right away. Instead, they just remember the transformations applied to some base dataset (e.g. a file). The transformations are only computed when an action requires a result to be returned to the driver program. This design enables Spark to run more efficiently – for example, we can realize that a dataset created through `map` will be used in a `reduce` and return only the result of the `reduce` to the driver, rather than the larger mapped dataset.

By default, each transformed RDD may be recomputed each time you run an action on it. However, you may also *persist* an RDD in memory using the `persist` (or `cache`) method, in which case Spark will keep the elements around on the cluster for much faster access the next time you query it. There is also support for persisting RDDs on disk, or replicated across multiple nodes.

Basics

[Scala](#)[Java](#)[Python](#)

To illustrate RDD basics, consider the simple program below:

```
val lines = sc.textFile("data.txt")
val lineLengths = lines.map(s => s.length)
val totalLength = lineLengths.reduce((a, b) => a + b)
```

The first line defines a base RDD from an external file. This dataset is not loaded in memory or otherwise acted on: `lines` is merely a pointer to the file. The second line defines `lineLengths` as the result of a `map` transformation. Again, `lineLengths` is *not* immediately computed, due to laziness. Finally, we run `reduce`, which is an action. At this point Spark breaks the computation into tasks to run on separate machines, and each machine runs both its part of the map and a local reduction, returning only its answer to the driver program.

If we also wanted to use `lineLengths` again later, we could add:

```
lineLengths.persist()
```

before the `reduce`, which would cause `lineLengths` to be saved in memory after the first time it is computed.

Passing Functions to Spark

[Scala](#)[Java](#)[Python](#)

Spark's API relies heavily on passing functions in the driver program to run on the cluster. There are two recommended ways to do this:

- [Anonymous function syntax](#), which can be used for short pieces of code.
- Static methods in a global singleton object. For example, you can define object `MyFunctions` and then pass `MyFunctions.func1`, as follows:

```
object MyFunctions {
  def func1(s: String): String = { ... }
}

myRdd.map(MyFunctions.func1)
```

Note that while it is also possible to pass a reference to a method in a class instance (as opposed to a singleton object), this requires sending the object that contains that class along with the method. For example, consider:

```
class MyClass {
  def func1(s: String): String = { ... }
  def doStuff(rdd: RDD[String]): RDD[String] = { rdd.map(func1) }
}
```

Here, if we create a new `MyClass` and call `doStuff` on it, the `map` inside there references the `func1` method *of that* `MyClass` *instance*, so the whole object needs to be sent to the cluster. It is similar to writing `rdd.map(x => this.func1(x))`.

In a similar way, accessing fields of the outer object will reference the whole object:

```
class MyClass {  
  val field = "Hello"  
  def doStuff(rdd: RDD[String]): RDD[String] = { rdd.map(x => field + x) }  
}
```

is equivalent to writing `rdd.map(x => this.field + x)`, which references all of `this`. To avoid this issue, the simplest way is to copy `field` into a local variable instead of accessing it externally:

```
def doStuff(rdd: RDD[String]): RDD[String] = {  
  val field_ = this.field  
  rdd.map(x => field_ + x)  
}
```

Working with Key-Value Pairs

[Scala](#)[Java](#)[Python](#)

While most Spark operations work on RDDs containing any type of objects, a few special operations are only available on RDDs of key-value pairs. The most common ones are distributed “shuffle” operations, such as grouping or aggregating the elements by a key.

In Scala, these operations are automatically available on RDDs containing [Tuple2](#) objects (the built-in tuples in the language, created by simply writing `(a, b)`), as long as you import `org.apache.spark.SparkContext._` in your program to enable Spark’s implicit conversions. The key-value pair operations are available in the [PairRDDFunctions](#) class, which automatically wraps around an RDD of tuples if you import the conversions.

For example, the following code uses the `reduceByKey` operation on key-value pairs to count how many times each line of text occurs in a file:

```
val lines = sc.textFile("data.txt")  
val pairs = lines.map(s => (s, 1))  
val counts = pairs.reduceByKey((a, b) => a + b)
```

We could also use `counts.sortByKey()`, for example, to sort the pairs alphabetically, and finally `counts.collect()` to bring them back to the driver program as an array of objects.

Note: when using custom objects as the key in key-value pair operations, you must be sure that a custom `equals()` method is accompanied with a matching `hashCode()` method. For full details, see the contract outlined in the [Object.hashCode\(\)](#) documentation.

Transformations

The following table lists some of the common transformations supported by Spark. Refer to the RDD API doc ([Scala](#), [Java](#), [Python](#)) and pair RDD functions doc ([Scala](#), [Java](#)) for details.

Transformation	Meaning
map (<i>func</i>)	Return a new distributed dataset formed by passing each element of the source through a function <i>func</i> .
filter (<i>func</i>)	Return a new dataset formed by selecting those elements of the source on which <i>func</i> returns true.
flatMap (<i>func</i>)	Similar to map, but each input item can be mapped to 0 or more output items (so <i>func</i> should return a Seq rather than a single item).
mapPartitions (<i>func</i>)	Similar to map, but runs separately on each partition (block) of the RDD, so <i>func</i> must be of type <code>Iterator<T> => Iterator<U></code> when running on an RDD of type T.
mapPartitionsWithIndex (<i>func</i>)	Similar to mapPartitions, but also provides <i>func</i> with an integer value representing the index of the partition, so <i>func</i> must be of type <code>(Int, Iterator<T>) => Iterator<U></code> when running on an RDD of type T.
sample (<i>withReplacement</i> , <i>fraction</i> , <i>seed</i>)	Sample a fraction <i>fraction</i> of the data, with or without replacement, using a given random number generator seed.
union (<i>otherDataset</i>)	Return a new dataset that contains the union of the elements in the source dataset and the argument.
intersection (<i>otherDataset</i>)	Return a new RDD that contains the intersection of elements in the source dataset and the argument.
distinct ([<i>numTasks</i>])	Return a new dataset that contains the distinct elements of the source dataset.
groupByKey ([<i>numTasks</i>])	When called on a dataset of (K, V) pairs, returns a dataset of (K, Iterable<V>) pairs. Note: If you are grouping in order to perform

an aggregation (such as a sum or average) over each key, using `reduceByKey` or `combineByKey` will yield much better performance.

Note: By default, the level of parallelism in the output depends on the number of partitions of the parent RDD. You can pass an optional `numTasks` argument to set a different number of tasks.

`reduceByKey(func, [numTasks])`

When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function *func*, which must be of type (V,V) => V. Like in `groupByKey`, the number of reduce tasks is configurable through an optional second argument.

`aggregateByKey(zeroValue)(seqOp, combOp, [numTasks])`

When called on a dataset of (K, V) pairs, returns a dataset of (K, U) pairs where the values for each key are aggregated using the given combine functions and a neutral "zero" value. Allows an aggregated value type that is different than the input value type, while avoiding unnecessary allocations. Like in `groupByKey`, the number of reduce tasks is configurable through an optional second argument.

`sortByKey([ascending], [numTasks])`

When called on a dataset of (K, V) pairs where K implements `Ordered`, returns a dataset of (K, V) pairs sorted by keys in ascending or descending order, as specified in the boolean `ascending` argument.

`join(otherDataset, [numTasks])`

When called on datasets of type (K, V) and (K, W), returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key. Outer joins are supported through `leftOuterJoin`, `rightOuterJoin`, and `fullOuterJoin`.

`cogroup(otherDataset, [numTasks])`

When called on datasets of type (K, V) and (K, W), returns a dataset of (K, Iterable<V>, Iterable<W>) tuples. This operation is also called `groupWith`.

`cartesian(otherDataset)`

When called on datasets of types T and U, returns a dataset of (T, U) pairs (all pairs of

elements).

pipe (<i>command</i> , [<i>envVars</i>])	Pipe each partition of the RDD through a shell command, e.g. a Perl or bash script. RDD elements are written to the process's stdin and lines output to its stdout are returned as an RDD of strings.
coalesce (<i>numPartitions</i>)	Decrease the number of partitions in the RDD to <i>numPartitions</i> . Useful for running operations more efficiently after filtering down a large dataset.
repartition (<i>numPartitions</i>)	Reshuffle the data in the RDD randomly to create either more or fewer partitions and balance it across them. This always shuffles all data over the network.
repartitionAndSortWithinPartitions (<i>partitioner</i>)	Repartition the RDD according to the given partitioner and, within each resulting partition, sort records by their keys. This is more efficient than calling <code>repartition</code> and then sorting within each partition because it can push the sorting down into the shuffle machinery.

Actions

The following table lists some of the common actions supported by Spark. Refer to the RDD API doc ([Scala](#), [Java](#), [Python](#)) and pair RDD functions doc ([Scala](#), [Java](#)) for details.

Action	Meaning
reduce (<i>func</i>)	Aggregate the elements of the dataset using a function <i>func</i> (which takes two arguments and returns one). The function should be commutative and associative so that it can be computed correctly in parallel.
collect ()	Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
count ()	Return the number of elements in the dataset.
first ()	Return the first element of the dataset (similar to <code>take(1)</code>).
take (<i>n</i>)	Return an array with the first <i>n</i> elements of the dataset. Note that this is currently not executed in parallel. Instead, the driver program computes all the elements.

takeSample (<i>withReplacement</i> , <i>num</i> , [<i>seed</i>])	Return an array with a random sample of <i>num</i> elements of the dataset, with or without replacement, optionally pre-specifying a random number generator seed.
takeOrdered (<i>n</i> , [<i>ordering</i>])	Return the first <i>n</i> elements of the RDD using either their natural order or a custom comparator.
saveAsTextFile (<i>path</i>)	Write the elements of the dataset as a text file (or set of text files) in a given directory in the local filesystem, HDFS or any other Hadoop-supported file system. Spark will call <code>toString</code> on each element to convert it to a line of text in the file.
saveAsSequenceFile (<i>path</i>) (Java and Scala)	Write the elements of the dataset as a Hadoop SequenceFile in a given path in the local filesystem, HDFS or any other Hadoop-supported file system. This is available on RDDs of key-value pairs that either implement Hadoop's Writable interface. In Scala, it is also available on types that are implicitly convertible to Writable (Spark includes conversions for basic types like <code>Int</code> , <code>Double</code> , <code>String</code> , etc).
saveAsObjectFile (<i>path</i>) (Java and Scala)	Write the elements of the dataset in a simple format using Java serialization, which can then be loaded using <code>sparkContext.objectFile()</code> .
countByKey ()	Only available on RDDs of type (K, V). Returns a hashmap of (K, Int) pairs with the count of each key.
foreach (<i>func</i>)	Run a function <i>func</i> on each element of the dataset. This is usually done for side effects such as updating an accumulator variable (see below) or interacting with external storage systems.

RDD Persistence

One of the most important capabilities in Spark is *persisting* (or *caching*) a dataset in memory across operations. When you persist an RDD, each node stores any partitions of it that it computes in memory and reuses them in other actions on that dataset (or datasets derived from it). This allows future actions to be much faster (often by more than 10x). Caching is a key tool for iterative algorithms and fast interactive use.

You can mark an RDD to be persisted using the `persist()` or `cache()` methods on it. The first time it is computed in an action, it will be kept in memory on the nodes. Spark's cache is fault-tolerant – if any partition of an RDD is lost, it will automatically be recomputed using the transformations that originally created it.

In addition, each persisted RDD can be stored using a different *storage level*, allowing you, for example, to persist the dataset on disk, persist it in memory but as serialized Java objects (to save

space), replicate it across nodes, or store it off-heap in [Tachyon](#). These levels are set by passing a `StorageLevel` object ([Scala](#), [Java](#), [Python](#)) to `persist()`. The `cache()` method is a shorthand for using the default storage level, which is `StorageLevel.MEMORY_ONLY` (store deserialized objects in memory). The full set of storage levels is:

Storage Level	Meaning
<code>MEMORY_ONLY</code>	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, some partitions will not be cached and will be recomputed on the fly each time they're needed. This is the default level.
<code>MEMORY_AND_DISK</code>	Store RDD as deserialized Java objects in the JVM. If the RDD does not fit in memory, store the partitions that don't fit on disk, and read them from there when they're needed.
<code>MEMORY_ONLY_SER</code>	Store RDD as <i>serialized</i> Java objects (one byte array per partition). This is generally more space-efficient than deserialized objects, especially when using a fast serializer , but more CPU-intensive to read.
<code>MEMORY_AND_DISK_SER</code>	Similar to <code>MEMORY_ONLY_SER</code> , but spill partitions that don't fit in memory to disk instead of recomputing them on the fly each time they're needed.
<code>DISK_ONLY</code>	Store the RDD partitions only on disk.
<code>MEMORY_ONLY_2</code> , <code>MEMORY_AND_DISK_2</code> , etc.	Same as the levels above, but replicate each partition on two cluster nodes.
<code>OFF_HEAP</code> (experimental)	Store RDD in serialized format in Tachyon . Compared to <code>MEMORY_ONLY_SER</code> , <code>OFF_HEAP</code> reduces garbage collection overhead and allows executors to be smaller and to share a pool of memory, making it attractive in environments with large heaps or multiple concurrent applications. Furthermore, as the RDDs reside in Tachyon, the crash of an executor does not lead to losing the in-memory cache. In this mode, the memory in Tachyon is discardable. Thus, Tachyon does not attempt to reconstruct a block that it evicts from memory.

Note: In Python, stored objects will always be serialized with the [Pickle](#) library, so it does not matter whether you choose a serialized level.

Spark also automatically persists some intermediate data in shuffle operations (e.g. `reduceByKey`), even without users calling `persist`. This is done to avoid recomputing the entire input if a node fails during the shuffle. We still recommend users call `persist` on the resulting RDD if they plan to reuse it.

Which Storage Level to Choose?

Spark's storage levels are meant to provide different trade-offs between memory usage and CPU efficiency. We recommend going through the following process to select one:

- If your RDDs fit comfortably with the default storage level (`MEMORY_ONLY`), leave them that way. This is the most CPU-efficient option, allowing operations on the RDDs to run as fast as possible.
- If not, try using `MEMORY_ONLY_SER` and [selecting a fast serialization library](#) to make the objects much more space-efficient, but still reasonably fast to access.
- Don't spill to disk unless the functions that computed your datasets are expensive, or they filter a large amount of the data. Otherwise, recomputing a partition may be as fast as reading it from disk.
- Use the replicated storage levels if you want fast fault recovery (e.g. if using Spark to serve requests from a web application). *All* the storage levels provide full fault tolerance by recomputing lost data, but the replicated ones let you continue running tasks on the RDD without waiting to recompute a lost partition.
- In environments with high amounts of memory or multiple applications, the experimental `OFF_HEAP` mode has several advantages:
 - It allows multiple executors to share the same pool of memory in Tachyon.
 - It significantly reduces garbage collection costs.
 - Cached data is not lost if individual executors crash.

Removing Data

Spark automatically monitors cache usage on each node and drops out old data partitions in a least-recently-used (LRU) fashion. If you would like to manually remove an RDD instead of waiting for it to fall out of the cache, use the `RDD.unpersist()` method.

Shared Variables

Normally, when a function passed to a Spark operation (such as `map` or `reduce`) is executed on a remote cluster node, it works on separate copies of all the variables used in the function. These variables are copied to each machine, and no updates to the variables on the remote machine are propagated back to the driver program. Supporting general, read-write shared variables across tasks would be inefficient. However, Spark does provide two limited types of *shared variables* for two common usage patterns: broadcast variables and accumulators.

Broadcast Variables

Broadcast variables allow the programmer to keep a read-only variable cached on each machine rather than shipping a copy of it with tasks. They can be used, for example, to give every node a copy of a large input dataset in an efficient manner. Spark also attempts to distribute broadcast variables using efficient broadcast algorithms to reduce communication cost.

Broadcast variables are created from a variable `v` by calling `sparkContext.broadcast(v)`. The broadcast variable is a wrapper around `v`, and its value can be accessed by calling the `value` method. The code below shows this:

Scala

Java

Python

```
scala> val broadcastVar = sc.broadcast(Array(1, 2, 3))
broadcastVar: org.apache.spark.broadcast.Broadcast[Array[Int]] = Broadcast(0)

scala> broadcastVar.value
res0: Array[Int] = Array(1, 2, 3)
```

After the broadcast variable is created, it should be used instead of the value `v` in any functions run on the cluster so that `v` is not shipped to the nodes more than once. In addition, the object `v` should not be modified after it is broadcast in order to ensure that all nodes get the same value of the broadcast variable (e.g. if the variable is shipped to a new node later).

Accumulators

Accumulators are variables that are only “added” to through an associative operation and can therefore be efficiently supported in parallel. They can be used to implement counters (as in MapReduce) or sums. Spark natively supports accumulators of numeric types, and programmers can add support for new types. If accumulators are created with a name, they will be displayed in Spark’s UI. This can be useful for understanding the progress of running stages (NOTE: this is not yet supported in Python).

An accumulator is created from an initial value `v` by calling `sparkContext.accumulator(v)`. Tasks running on the cluster can then add to it using the `add` method or the `+=` operator (in Scala and Python). However, they cannot read its value. Only the driver program can read the accumulator’s value, using its `value` method.

The code below shows an accumulator being used to add up the elements of an array:

Scala

Java

Python

```
scala> val accum = sc.accumulator(0, "My Accumulator")
accum: spark.Accumulator[Int] = 0

scala> sc.parallelize(Array(1, 2, 3, 4)).foreach(x => accum += x)
...
10/09/29 18:41:08 INFO SparkContext: Tasks finished in 0.317106 s

scala> accum.value
res2: Int = 10
```

While this code used the built-in support for accumulators of type `Int`, programmers can also create their own types by subclassing [AccumulatorParam](#). The `AccumulatorParam` interface has two methods: `zero` for providing a “zero value” for your data type, and `addInPlace` for adding two values together. For example, supposing we had a `Vector` class representing mathematical vectors, we could write:

```
object VectorAccumulatorParam extends AccumulatorParam[Vector] {  
  def zero(initialValue: Vector): Vector = {  
    Vector.zeros(initialValue.size)  
  }  
  def addInPlace(v1: Vector, v2: Vector): Vector = {  
    v1 += v2  
  }  
}  
  
// Then, create an Accumulator of this type:  
val vecAccum = sc.accumulator(new Vector(...))(VectorAccumulatorParam)
```

In Scala, Spark also supports the more general [Accumulable](#) interface to accumulate data where the resulting type is not the same as the elements added (e.g. build a list by collecting together elements), and the `sparkContext.accumulableCollection` method for accumulating common Scala collection types.

For accumulator updates performed inside **actions only**, Spark guarantees that each task’s update to the accumulator will only be applied once, i.e. restarted tasks will not update the value. In transformations, users should be aware of that each task’s update may be applied more than once if tasks or job stages are re-executed.

Deploying to a Cluster

The [application submission guide](#) describes how to submit applications to a cluster. In short, once you package your application into a JAR (for Java/Scala) or a set of `.py` or `.zip` files (for Python), the `bin/spark-submit` script lets you submit it to any supported cluster manager.

Unit Testing

Spark is friendly to unit testing with any popular unit test framework. Simply create a `sparkContext` in your test with the master URL set to `local`, run your operations, and then call `sparkContext.stop()` to tear it down. Make sure you stop the context within a `finally` block or the test framework’s `tearDown` method, as Spark does not support two contexts running concurrently in the same program.

Migrating from pre-1.0 Versions of Spark

[Scala](#)[Java](#)[Python](#)

Spark 1.0 freezes the API of Spark Core for the 1.X series, in that any API available today that is not marked “experimental” or “developer API” will be supported in future versions. The only change for Scala users is that the grouping operations, e.g. `groupByKey`, `cogroup` and `join`, have changed from returning `(key, Seq[value])` pairs to `(key, Iterable[value])`.

Migration guides are also available for [Spark Streaming](#), [MLlib](#) and [GraphX](#).

Where to Go from Here

You can see some [example Spark programs](#) on the Spark website. In addition, Spark includes several samples in the `examples` directory ([Scala](#), [Java](#), [Python](#)). You can run Java and Scala examples by passing the class name to Spark’s `bin/run-example` script; for instance:

```
./bin/run-example SparkPi
```

For Python examples, use `spark-submit` instead:

```
./bin/spark-submit examples/src/main/python/pi.py
```

For help on optimizing your programs, the [configuration](#) and [tuning](#) guides provide information on best practices. They are especially important for making sure that your data is stored in memory in an efficient format. For help on deploying, the [cluster mode overview](#) describes the components involved in distributed operation and supported cluster managers.

Finally, full API documentation is available in [Scala](#), [Java](#) and [Python](#).